

Цикл while

Как уже было сказано в предыдущем уроке существуют две основные разновидности цикла:

- циклы, повторяющиеся определенное количество раз (for, счетные циклы, **counting loops**);
- циклы, повторяющиеся до наступления определенного события (while, условные циклы, **conditional loops**)

Иногда нужно, чтобы цикл выполнялся до наступления некоторого события, и количество итераций в этом случае заранее оценить просто невозможно. И здесь на помощь приходит цикл `while`.

Структура цикла `while` в Python выглядит так:

```
while условие:
```

```
    блок кода
```

Двоеточие (:) в конце строки с инструкцией `while` сообщает Python, что дальше находится **блок команд**. В блок входят все строки, расположенные с отступом от строки с инструкцией `while`, вплоть до следующей строки без отступа.

Блок команд, который выполняется в цикле `while`, называется **телом цикла**.

Рассмотрим код, использующий цикл `while`, который распечатает 10 раз слово `Привет`:

```
i = 0

while i < 10:

    print('Привет')

    i += 1
```

Такой код можно легко заменить циклом `for`, поскольку мы заранее знаем сколько раз нужно выполнить тело цикла. Однако так бывает не всегда.

Напишем программу, которая считывает числа и выводит их квадраты, пока не будет введено -1. При такой постановке задачи мы не можем воспользоваться циклом `for`, так как не знаем сколько чисел будет предшествовать числу -1.

```
num = int(input())

while num != -1:

    print('Квадрат вашего числа равен:', num * num)
```

```
num = int(input())
```

В качестве начального значения переменной `num`, мы используем первое из чисел. Далее пока выполняется условие цикла, а именно, пока введенное число не равно -1, мы исполняем тело цикла. В тело цикла входит две команды:

1. напечатать квадрат введенного числа;
2. считать следующее число.

Важным являются два момента:

1. правильная инициализация переменной `num`;
2. изменение переменной `num` внутри цикла `while`.

Важно: если не изменять переменную `num` внутри цикла, то можно получить так называемый **бесконечный цикл**, который будет выполняться бесконечно много раз.

Цикл `while` очень похож на условный оператор `if`. Разница заключается в том, что в случае с условным оператором соответствующий блок кода будет выполняться только один раз, тогда как с циклом `while` блок кода будет выполнен многократно.

Считывание данных до стоп значения

Часто при решении задач на цикл `while`, мы считываем данные, до тех пор пока пользователь не введет некоторое значение, которое называют **стоп значением**. Напишем программу, которая считывает числа и находит их сумму, до тех пор пока пользователь не введет слово `stop`:

```
text = input()

total = 0

while text != 'stop':

    num = int(text)

    total += num

    text = input()

print('Сумма чисел равна', total)
```

Такой код будет часто использоваться при решении задач.

Бесконечный цикл

Всегда, кроме редких случаев, цикл `while` должен содержать возможность завершиться. То есть в цикле что-то должно сделать проверяемое условие

ложным. Если цикл не имеет возможности завершиться, то он называется **бесконечным циклом**. Бесконечный цикл продолжает повторяться до тех пор, пока программа не будет прервана. Бесконечные циклы обычно появляются, когда программист забывает написать программный код внутри цикла, который делает проверяемое условие ложным. В большинстве случаев следует избегать применения бесконечных циклов.

Пример бесконечного цикла:

```
i = 0

total = 0

while i < 10:

    total += i
```

Так как в теле цикла не происходит изменения переменной `i`, то условие `i < 10` остается истинным и цикл выполняется бесконечно много раз.

Бесконечные циклы можно использовать в связке с оператором прерывания `break`. Об этом будет рассказано в следующих уроках.

Примечания

Примечание 1. Цикл `while` получил свое название из-за характера своей работы: он выполняет некую задачу до тех пор, **пока** условие является истинным. Слово *while* на английском означает как раз "пока".

Примечание 2. Цикл `while` называют циклом с **предусловием**, поскольку выполнению тела цикла предшествует проверка условия (сначала проверяется условие, а уже затем выполняется тело цикла).

Примечание 3. Однократное выполнение тела цикла называется **итерацией** цикла.

Примечание 4. Цикл `while` может не выполниться ни одного раза. Например, следующий код:

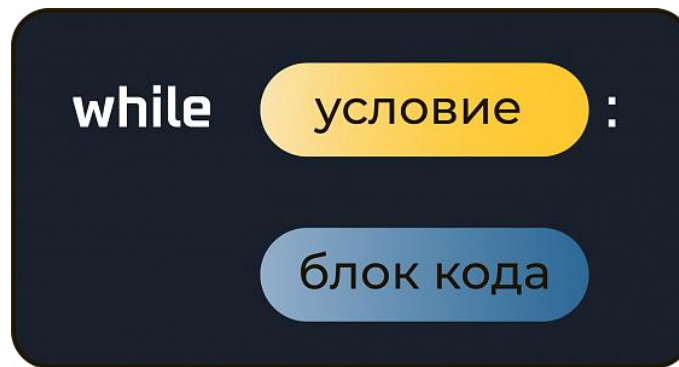
```
i = -1

while i > 0:

    print('Hello world!')
```

не выведет текст, поскольку условие `i > 0` ложно с самого начала.

Примечание 5. Графическое представление цикла `while` имеет вид:



Примечание 6. Условие в цикле `while`, как и в условном операторе `if`, может содержать логические операции `or`, `and`, `not`.

Обработка цифр числа

При изучении целых чисел (тип данных `int`), мы говорили про операцию целочисленного деления `//` и операцию нахождения остатка от деления одного целого числа на другое `%`. Используя цикл `while` и две данных операции, можно обработать цифры числа с произвольным количеством разрядов (цифр).

Пусть дано натуральное число `n`. Тогда:

- результатом операции `n % 10` – является последняя цифра числа;
- результатом операции `n // 10` – является число с удаленной последней цифрой.

Напишем программу, которая считывает натуральное число (целое положительное) и обрабатывает его цифры.

```
n = int(input())

while n != 0: # пока в числе есть цифры

    last_digit = n % 10 # получить последнюю цифру

    # код обработки последней цифры

    n = n // 10 # удалить последнюю цифру из числа
```

Цикл `while` работает до тех пор пока в числе есть необработанные цифры. Тело цикла содержит:

1. процедуру получения последней цифры `last_digit = n % 10`;
2. код обработки последней цифры;
3. процедуру удаления последней цифры из числа `n = n // 10`.

В качестве процедуры обработки может быть все, что угодно: вывод цифр, нахождение суммы, произведения цифр, нахождение наибольшей или наименьшей цифры, подсчет цифр удовлетворяющих некоторому условию и т.д.

Напишем программу, которая определяет есть ли в числе цифра 7.

```
num = int(input())

has_seven = False # сигнальная метка

while num != 0:

    last_digit = num % 10

    if last_digit == 7:

        has_seven = True

    num = num // 10

if has_seven == True:

    print('YES')

else:

    print('NO')
```